

spinalSynth: Implementation Specification

Table of Contents

- 1. spinalHDL Hierarchy
- 2. Synth (System Top) Module
- 3. TimingGenerator
- 4. Uart & Registers
 - 4.1 UartRx
 - 4.2 UartProtocolDecoder
 - 4.3 RegisterBank
- 5. Oscillator
 - 5.1 Oscillator Submodules
 - 5.2 Oscillator signal flow
- 6. Envelope Generator
 - 6.1 EnvelopeGenerator (Top-Level Wrapper)
 - 6.2 EnvelopeCtrl
 - 6.3 EnvelopeAccumulator
 - 6.4 EnvelopeShaper
 - 6.5 Signal Flow & Pipeline Timing
- 7. Attenuator
- 8. Decimator
- 9. I2STransmitter
 - 9.1 Gated Startup and Idle State

1. spinalHDL Hierarchy

The spinalHDL implementation shall use the following hierarchy for folders, subfolders and files.

```
spinalSynth/
├─ build.sbt                # SBT build configuration (dependencies, Scala version)
├─ project/                 # SBT plumbing
│  └─ build.properties      # Defines the SBT version
├─ src/
│  └─ main/
│     └─ scala/
│        └─ synth/          # Root package for the project
│           └─ Synth.scala   # Top-level System Integration (per Section
2)
│           └─ Main.scala    # Hardware generation entry point (Verilog
generator)
│              └─ common/    # Shared system types and bundles
│                 └─ Types.scala # Unified hardware types and bundles
│                 └─ timing/  # System control and tick generation
│                    └─ TimingGenerator.scala # Tick generation logic (per Section 3)
│                    └─ uart/ # Control Path logic
│                       └─ Uart.scala      # UART Subsystem Wrapper
│                       └─ UartRx.scala    # UART Receiver
│                       └─ UartProtocolDecoder.scala # Protocol Parser
│                       └─ RegisterBank.scala # Parameter Storage
│                       └─ oscillator/     # Core Oscillator logic (per Section 4)
│                          └─ Oscillator.scala # Main Oscillator module
│                          └─ Accumulator.scala # Phase logic
│                          └─ Noise.scala    # LFSR logic
```

wrapping

- **Decimator**: Downsamples the 480 kHz audio sample stream to 48 kHz.
 - **I2STransmitter**: Serializes parallel audio samples into a stereo I2S bitstream.
- **Clock & Reset Distribution**: Run all submodules synchronously inside a single 24 MHz clocking area with asynchronous active-high reset active.
- **Register Pipeline Latency Alignment**: The cascade of two series registered attenuators introduces exactly **2 clock cycles** of registered pipeline latency between the oscillator and the decimator. The clock-enable decimated `sampleTick` is delayed by exactly 2 cycles (`Delay(timingGen.io.sampleTick, cycleCount = 2)`) to keep the Decimator perfectly aligned with the audio data stream.
- **Subsystem Isolation**: Keep Synth strictly as an abstraction/integration layer containing no DSP math details or physical protocol implementation details.

Clocking and Reset

The system operates within a single clock domain managed at the top level:

- **Clock**: 24 MHz external input.
- **Reset**: Asynchronous, Active-High.

IO Bundle

```
val io = new Bundle {
    val clk24MHz = in Bool()
    val reset    = in Bool()

    val uartRx   = in Bool()

    val i2sBclk  = out Bool()
    val i2sLrclk = out Bool()
    val i2sData  = out Bool()
}
```

3. TimingGenerator

IO Bundle

```
val io = new Bundle {
    val phaseTick = out Bool()
    val sampleTick = out Bool()
}
```

Internal Structure

Signal	Width	Purpose
phaseCounter	6 bit	modulo-50 counter
sampleCounter	9 bit	modulo-500 counter

Both counters:

- operate directly from the 24 MHz master clock
- run independently
- are free-running

Tick Behaviour

Both tick outputs:

- are registered
- synchronous
- one clock cycle wide
- default to **False**

4. Uart Subsystem

This subsystem encapsulates serial data decoding, command protocol framing, and parameter register storage into a single cohesive module.

Submodule Structure

```
Uart
├─ UartRx
├─ UartProtocolDecoder
└─ RegisterBank
```

IO Bundle

```
val io = new Bundle {
    val rx      = in Bool()
    val config  = out(OscillatorConfig())
    val envConfig = out(EnvelopeConfig())
}
```

4.1 UartRx

Purpose: Converts the serial UART bitstream into parallel 8-bit bytes. It operates at 115,200 baud using a 208-tick bit period and includes start-bit verification.

IO Bundle

```
val io = new Bundle {
    val rx      = in Bool()
    val byteOut = master(Flow(Bits(8 bits)))
}
```

4.2 UartProtocolDecoder

Purpose: Frames individual bytes into 3-byte command packets: `[Address/Command]`, `[Data]`, and `[Reserved]`. It asserts `writeEnable` only when a full frame is valid.

IO Bundle

```
val io = new Bundle {
    val rxByte    = slave(Flow(Bits(8 bits)))
    val regWrite = master(Flow(RegisterWrite()))
}
```

4.3 RegisterBank

Purpose: Stores the current state of the synthesizer parameters. It implements an atomic update for the 24-bit frequency word, ensuring all three bytes are applied simultaneously to the DDS engine upon writing to the high-byte address.

Atomic Write Staging Mechanism

To prevent audibly jarring sweep glitches or frequency artifacts, the multi-byte frequency configuration transitions atomically:

- **FREQ_LOW (0x00):** Staged into a temporary staging register `freqLowShadow`.
- **FREQ_MID (0x01):** Staged into a temporary staging register `freqMidShadow`.
- **FREQ_HIGH (0x02):** Directly commits the newly written high byte (`freqHighReg`) and transfers both staged shadow registers (`freqMidReg := freqMidShadow`, `freqLowReg := freqLowShadow`) to the active registers simultaneously on a single clock edge.

IO Bundle

```
val io = new Bundle {
    val regWrite = slave(Flow(RegisterWrite()))
    val config   = out(OscillatorConfig())
    val envConfig = out(EnvelopeConfig())
}
```

5. Oscillator

The Oscillator is a module that connects the four submodules, as shown in the following internal submodule structure. It serves as an abstraction layer above the generation of the oscillating audio signals.

Purpose

Core DDS synthesis engine.

Responsibilities:

- phase accumulation
- waveform generation
- noise generation
- mux between waveforms and noise
- oversampled sample generation

IO Bundle

```
val io = new Bundle {
  val phaseTick = in Bool()
  val config    = in(OscillatorConfig())
  val sample    = master(Flow(SInt(16 bits)))
}
```

Internal structure of submodules:

```
Oscillator
├─ Accumulator
├─ Noise
├─ Generators
└─ Mux
```

5.1 Oscillator Submodules

Accumulator

Responsible for:

- phase register
- phase accumulation
- frequency addition

```
val io = new Bundle {
  val phaseTick = in Bool()
  val freqWord  = in UInt(24 bits)
  val phase     = out UInt(24 bits)
}
```

Noise

Responsible for:

- LFSR register
- feedback logic
- shift/update logic
- noise sample generation

```
val io = new Bundle {
  val phaseTick = in Bool()
  val sample    = out SInt(16 bits)
}
```

Generators

Responsible for:

- saw generation
- square generation
- PWM generation
- triangle generation

```
val io = new Bundle {
  val phase    = in UInt(24 bits)
  val pwmWidth = in UInt(8 bits)
  val waves    = out(Waveforms())
}
```

The module shall contain only combinational logic.

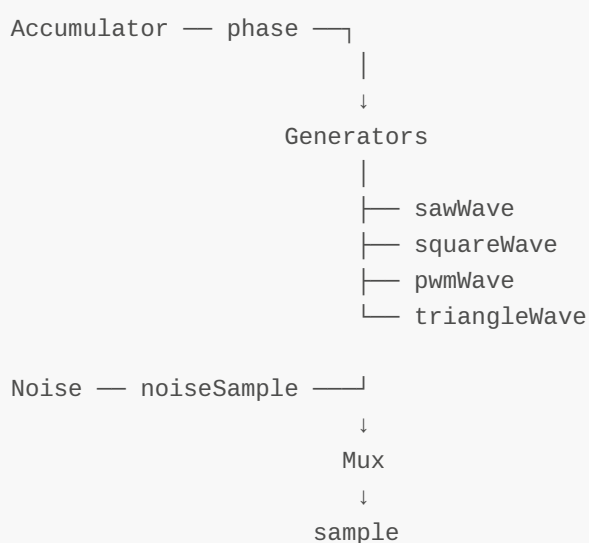
Mux

Responsible for:

- waveform or noise selection
- final waveform routing

```
val io = new Bundle {
  val waveSelect = in UInt(3 bits)
  val waves      = in(Waveforms())
  val noiseWave  = in SInt(16 bits)
  val sample     = out SInt(16 bits)
}
```

5.2 Oscillator signal flow



6. Envelope Generator

6.1 EnvelopeGenerator (Top-Level Wrapper)

Purpose

The `EnvelopeGenerator` top-level wrapper acts as the coordinator. It encapsulates the three core submodules (`EnvelopeCtrl`, `EnvelopeAccumulator`, and `EnvelopeShaper`), binds them together, registers parameters to the global register map, and packages the outputs into a synced flow rate.

IO Bundle

```
class EnvelopeGenerator extends Component {
  val io = new Bundle {
    // Clock Heartbeat & Sync inputs
    val phaseTick = in Bool() // Heartbeat tick synced with 480 kHz
    audio rate
    val syncIn    = in Bool() // External trigger for Hard or Soft Sync
    val midiClock = in Bool() // External MIDI clock tick (24 PPQN
    pulse)
    val config    = in(EnvelopeConfig()) // Packaged register configurations

    // System Outputs
    val envelopeOut      = master(Flow(UInt(10 bits))) // Unipolar output (0 to 1023)
    val envelopeOutSigned = master(Flow(SInt(10 bits))) // Bipolar output (-512 to
+511)
  }
}
```

Internal Architecture

The wrapper instantiates the submodules and wires their control signals. The outputs from the `EnvelopeShaper` are driven directly to the system audio/control busses.

```
val ctrl          = new EnvelopeCtrl()
val accumulator   = new EnvelopeAccumulator()
val shaper        = new EnvelopeShaper()

// Connecting Ctrl to Accumulator
accumulator.io.resetAccum := ctrl.io.resetAccum
accumulator.io.runAccum   := ctrl.io.runAccum
accumulator.io accumDir   := ctrl.io accumDir
accumulator.io.phaseInc   := ctrl.io.phaseInc
ctrl.io.segmentDone      := accumulator.io.segmentDone

// Connecting Accumulator and Ctrl to Shaper
shaper.io.phaseTick      := io.phaseTick
shaper.io.baseIndex      := accumulator.io.baseIndex
shaper.io.fraction       := accumulator.io.fraction
shaper.io.curveSelect    := ctrl.io.curveSelect
shaper.io.sustainLevel   := io.config.sustain
shaper.io.activeStage    := ctrl.io.activeStage

// Top-Level Outputs
io.envelopeOut      <=> shaper.io.envelopeOut
io.envelopeOutSigned <=> shaper.io.envelopeOutSigned
```


6.2 EnvelopeCtrl

Purpose

The `EnvelopeCtrl` submodule serves as the "brain". It is responsible for parsing control registers, driving the ADSR playback state machine, computing direction and synchronization triggers, and fetching stage-appropriate increment values from a static lookup table.

IO Bundle

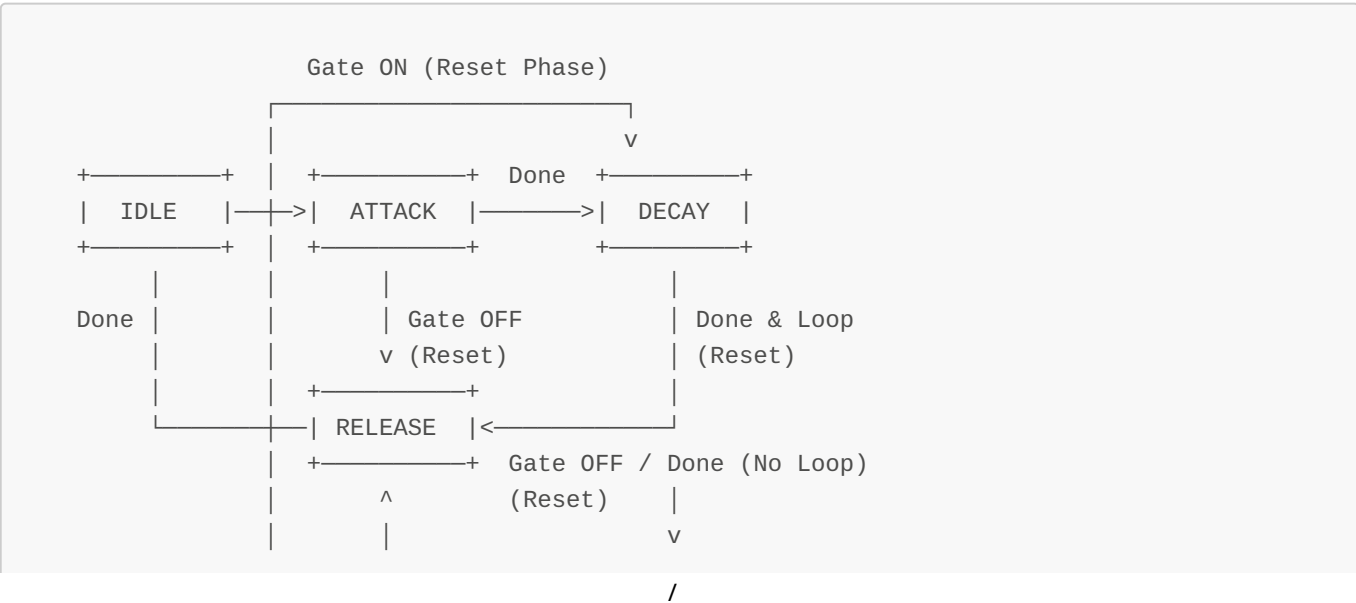
```
class EnvelopeCtrl extends Component {
  val io = new Bundle {
    // Inputs from Top Wrapper
    val syncIn      = in Bool()
    val midiClock    = in Bool()
    val config       = in(EnvelopeConfig())
    val segmentDone  = in Bool()           // High when accumulator sweeps to target
    limit

    // Outputs to Accumulator
    val resetAccum   = out Bool()
    val runAccum     = out Bool()
    val accumDir     = out Bool()         // 0 = Forward, 1 = Reverse
    val phaseInc     = out UInt(22 bits)

    // Outputs to Shaper
    val curveSelect  = out UInt(2 bits)
    val activeStage  = out UInt(3 bits)  // IDLE=0, ATTACK=1, DECAY=2, SUSTAIN=3,
    RELEASE=4
  }
}
```

State Machine Design

The ADSR engine is implemented as a built-in SpinalHDL `StateMachine` (`spinal.lib.fsm`).





It controls the envelope stages using five main states:

- **IDLE (Stage 0):** The default idle state. A **Gate ON** trigger resets the accumulator and transitions the FSM to **ATTACK**.
- **ATTACK (Stage 1):** The phase accumulator counts forward. If **Gate OFF** is detected, it transitions to **RELEASE**. When the segment completes (accumulator hits 1023), it resets the accumulator and transitions to **DECAY**.
- **DECAY (Stage 2):** The phase accumulator counts forward. If **Gate OFF** is detected, it transitions to **RELEASE**. When the segment completes, it transitions to **SUSTAIN** (or loops back to **ATTACK** if Looping/LFO mode is active).
- **SUSTAIN (Stage 3):** The accumulator is paused, holding the output at the configured sustain level. A **Gate OFF** trigger resets the accumulator and transitions the FSM to **RELEASE**.
- **RELEASE (Stage 4):** The phase accumulator counts forward. If **Gate ON** is triggered, it resets the accumulator and transitions to **ATTACK**. When the segment completes, it transitions back to **IDLE**.

Playback & Sync Controller

- **Looping (LFO Mode):** When `config.ctrl[2]` (Loop Enable) is active, transitioning out of **DECAY** loops instantly back to **ATTACK** instead of going to **SUSTAIN**.
- **Reverse Mode:** When `config.ctrl[4]` is active, the accumulator direction is inverted (`accumDir := True`), altering the counting sequence.
- **Ping-Pong Mode:** When `config.ctrl[3]` is active, a forward segment completion triggers a reverse segment immediately on the same stage before transitioning states.
- **Sync Logic:**
 - **Hard Sync:** A rising edge on `syncIn` forces the FSM back to **ATTACK** and sets `resetAccum := True`.
 - **MIDI Sync:** When enabled, the phase increment `phaseInc` is scaled according to incoming `midiClock` ticks rather than the default time register mapping.

Logarithmic Time-to-Increment Lookup Table (ROM)

Calculating the logarithmic time-duration mapping at runtime requires expensive divisor blocks. To ensure ASIC portability, the 256 increment coefficients are computed in Scala at compile-time and instantiated as a static hardware ROM (`Mem` in SpinalHDL).

Scala ROM Calculator Formula:

```
val clockFreq = 24000000.0 // 24 MHz
val tMin      = 0.0005     // 0.5 ms
val tMax      = 30.0       // 30.0 s

val lutContent = for (p <- 0 until 256) yield {
  val t = tMin * math.pow(tMax / tMin, p / 255.0)
  val inc = math.round((math.pow(2, 32) / (t * clockFreq)))
  U(inc, 22 bits)
}

val rom = Mem(UInt(22 bits), 256) init(lutContent)
```

6.3 EnvelopeAccumulator

Purpose

The **EnvelopeAccumulator** is a high-speed 32-bit digital register that acts as the phase counter. It increments on every 24 MHz system clock cycle when enabled, driving the envelope's progress through time.

IO Bundle

```
class EnvelopeAccumulator extends Component {
  val io = new Bundle {
    // Inputs from Control Unit
    val resetAccum = in Bool()
    val runAccum   = in Bool()
    val accumDir   = in Bool()           // 0 = Forward (Up), 1 = Reverse (Down)
    val phaseInc   = in UInt(22 bits)

    // Outputs
    val segmentDone = out Bool()         // Boundary completion pulse
    val baseIndex   = out UInt(8 bits)   // LUT address (Upper 8 integer bits)
    val fraction    = out UInt(2 bits)   // Interpolation fraction (Lower 2
integer bits)
  }
}
```

Counter & Overflow Behavior

- **Accumulator register:** `val accum = Reg(UInt(32 bits)) init(0)`
- **Accumulation Logic:**
 - If `io.resetAccum` is asserted, reset `accum := 0`.
 - If `io.runAccum` is High:
 - If `io.accumDir` is Forward (`False`), `accum := accum + io.phaseInc`.
 - If `io.accumDir` is Reverse (`True`), `accum := accum - io.phaseInc`.
- **Output Splitting:**
 - `io.baseIndex := accum(31 downto 24)`
 - `io.fraction := accum(23 downto 22)`
- **Segment Boundaries & Done Detection:**
 - In **Forward Mode**, completion is hit when the accum register overflows (wraps past 32-bit maximum).
 - In **Reverse Mode**, completion is hit when the accum register underflows (wraps past 0).
 - `io.segmentDone := (Forward && overflow) || (Reverse && underflow)`

6.4 EnvelopeShaper

Purpose

The **EnvelopeShaper** transforms the raw, linear accumulator output into customized, musically natural curves. It reads two consecutive points from a 257-entry curve ROM (Lin, Exp, Log, S-Curve) based on the 8-bit Base Index, performs linear interpolation in pure multiplierless combinational logic using the 2-bit fraction, and outputs unipolar/bipolar audio-rate flows.

IO Bundle

```
class EnvelopeShaper extends Component {
  val io = new Bundle {
```

```

    val phaseTick      = in Bool()           // Gating tick
    val baseIndex      = in UInt(8 bits)
    val fraction       = in UInt(2 bits)
    val curveSelect    = in UInt(2 bits)     // 00=Lin, 01=Exp, 10=Log, 11=S-Curve
    val sustainLevel   = in UInt(8 bits)
    val activeStage    = in UInt(3 bits)

    // Output flows
    val envelopeOut     = master(Flow(UInt(10 bits)))
    val envelopeOutSigned = master(Flow(SInt(10 bits)))
  }
}

```

ROM Curves

The shaper houses four distinct compile-time calculated ROMs, each with **257 entries** to safely compute lookup boundary intervals `LUT[x+1]` when `x = 255` without dynamic wrapping checks.

- **Linear ROM:**  $= x * (255.0 / 255.0)$ (0 to 255)
- **Exponential ROM:** Pre-calculated with decaying inverse-charge capacitor curves.
- **Logarithmic ROM:** Pre-calculated with fast initial rise profiles.
- **S-Curve (Sigmoid) ROM:** Pre-calculated with ease-in/ease-out cosine profiles.

```

val linRom = Mem(UInt(8 bits), 257) init(linContent)
val expRom = Mem(UInt(8 bits), 257) init(expContent)
val logRom = Mem(UInt(8 bits), 257) init(logContent)
val sigRom = Mem(UInt(8 bits), 257) init(sigContent)

```

Multiplierless Hybrid 8+2 Linear Interpolation

Linear interpolation requires the formula: $Y = Y_0 + (f / 4) * (Y_1 - Y_0)$. To prevent expensive synthesis of physical multipliers on custom silicon, the fraction multiplication $(f/4) * \text{delta_Y}$ is resolved in combinational shift-add structures based on the 2 fractional bits:

```

val y0 = UInt(8 bits)
val y1 = UInt(8 bits)

// Multiplexer select based on curveSelect register
y0 := curveMux(io.curveSelect, io.baseIndex)
y1 := curveMux(io.curveSelect, io.baseIndex + 1)

// Safely zero-extend to 9-bit SInt to prevent signed casting MSB sign-bit bugs
val y0Signed = y0.intoSInt
val y1Signed = y1.intoSInt

// Compute signed delta: Y1 - Y0 (10-bit signed SInt)
val delta = y1Signed - y0Signed

// interp holds (Y0 * 4 + f * delta) inside a safe 12-bit signed width to avoid overflow
val interp = SInt(12 bits)

```

```
// Pre-shifted terms for clean hardware synthesis
val y0Shifted    = (y0Signed << 2).resize(12 bits) // Y0 * 4
val deltaShifted = (delta << 1).resize(12 bits)   // 2 * delta
val deltaResized = delta.resize(12 bits)

switch(io.fraction) {
  is(0) { interp := y0Shifted }
  is(1) { interp := y0Shifted + deltaResized }
  is(2) { interp := y0Shifted + deltaShifted }
  is(3) { interp := y0Shifted + deltaShifted + deltaResized }
}

// Convert to 10-bit unsigned unipolar output (0 to 1023)
val finalValUnipolar = interp.asUInt.resize(10 bits)
```

Parallel Bipolar Output Flow:

- **Unipolar Flow:** `io.envelopeOut.payload := finalValUnipolar`
- **Bipolar Flow:** Bypasses arithmetic blocks by shifting the unipolar range (0 to 1023) to signed (-512 to +511) through direct inversion of the unipolar MSB:

```
io.envelopeOutSigned.payload := (finalValUnipolar ^ 0x200).asSInt
```

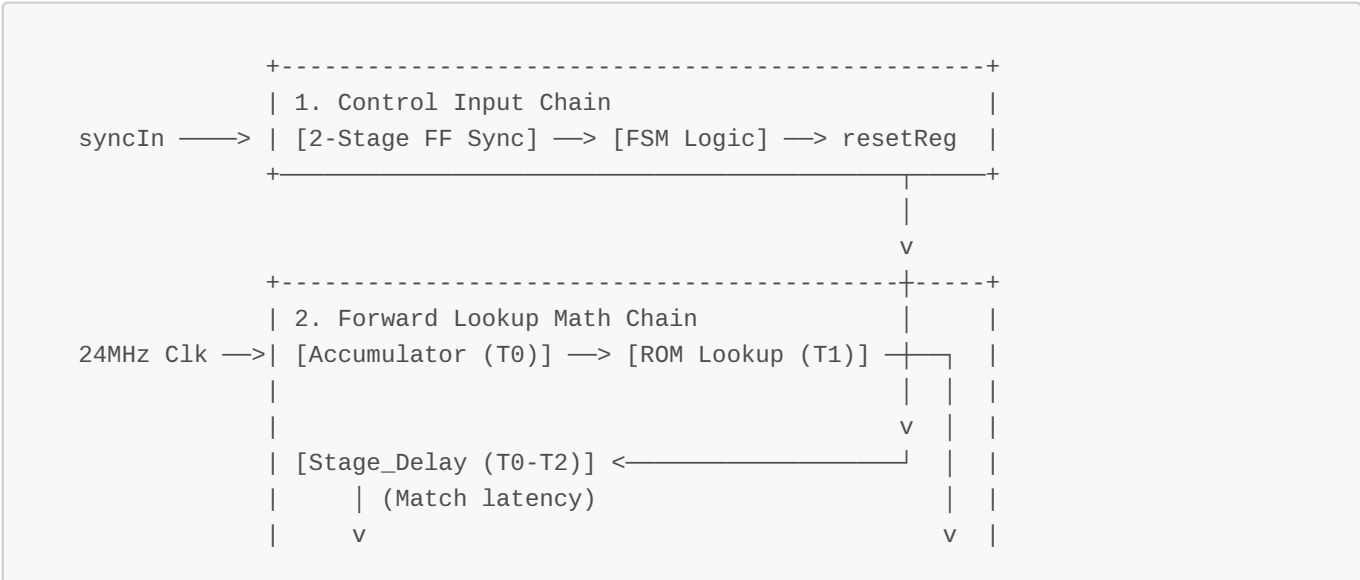
- **Heartbeat Synchronizer:**

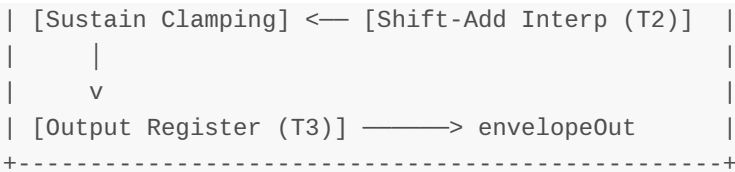
```
io.envelopeOut.valid      := io.phaseTick
io.envelopeOutSigned.valid := io.phaseTick
```

6.5 Signal Flow & Pipeline Timing

To ensure hardware timing closure at 24 MHz and robust, glitch-free control transitions, the Envelope Generator architecture isolates data lookup, mathematical computation, and control registers into dedicated synchronous pipeline stages.

The design implements three distinct hardware signal chains:





6.5.1 Chain 1: Control Input Propagation (syncIn, Gate -> Accumulator)

This path synchronizes asynchronous external controls and propagates internal register signals to steer the accumulator:

- **External Sync Input (syncIn):** Connects to a standard 2-stage flip-flop synchronizer clocked at 24 MHz to prevent metastability.
 - Latency: **2 clock cycles** (synchronization penalty).
- **Register Settings (config):** Synchronous from the RegisterBank.
- **State Updates:** The FSM processes inputs combinatorially and issues resetAccum or runAccum control registers to the accumulator on the next cycle.
 - Total Control Latency: **3 clock cycles** for syncIn, **1 clock cycle** for register-driven gate triggers.

6.5.2 Chain 2: Forward Lookup Math Pipeline (Accumulator -> Output)

This is the core mathematical flow designed to maintain a 24 MHz clock cycle bound through registered cells:

- **T0 (Accumulation Stage):** The 32-bit register accum increments by phaseInc. The split base index and fraction bits are output stable.
- **T1 (ROM Lookup Stage):** The 8-bit index addresses the dual boundary ports LUT[x] and LUT[x+1]. The memory array lookup requires **1 clock cycle** to fetch and settle output registers.
- **T2 (Arithmetic Stage):** The combinational multiplexed shift-add interpolator evaluates $Y = Y0 + (f/4) * \Delta_Y$ instantly.
- **T3 (Output Stage):** To isolate critical routing paths, the final unipolar and bipolar amplitude values are registered to the physical output ports and qualified with phaseTick.
 - Total Forward Latency: **3 clock cycles** (fully balanced).

6.5.3 Chain 3: Sustain Stage Delay Matching (sustain -> Output Clamping)

During the SUSTAIN state, the envelope holds its output at the static sustain value rather than accumulating with the phaseInc.

- **The Early Transition Problem:** Because the math pipeline has a **3-cycle latency**, if the FSM immediately switches the output to the static sustainLevel1 value upon entering the SUSTAIN state, the output port will jump to the sustain value **3 cycles too early**, truncating the final decay data points still traversing the pipeline!
- **The Solution (Delay Matching):** The active state indicators (like activeStage and FSM state controls) are routed through a **3-stage shift register pipeline (Stage_Delay)** inside the Shaper. This delays the sustain clamping multiplexer switch by exactly **3 cycles**, matching the lookup pipeline latency and ensuring a perfectly seamless, glitch-free decay-to-sustain transition.

7. Attenuator

Purpose

Applies volume scaling and attenuation dynamically on the oversampled 480 kHz audio sample stream.

Parameterization

The module is parameterized at compile-time to support dynamic input volume register sizes:

- **volumeWidth** (**Int**, default = 8): Defines the bit-width of the volume control input.

IO Bundle

```
class Attenuator(volumeWidth: Int = 8) extends Component {
  val io = new Bundle {
    val sampleIn  = slave(Flow(SInt(16 bits)))
    val volume    = in  UInt(volumeWidth bits)
    val sampleOut = master(Flow(SInt(16 bits)))
  }
}
```

Mathematical Scaling

To prevent sign-bit alignment errors during dynamic multiplication:

1. Zero-extend **io.volume** to a signed integer (**volumeSigned** = **io.volume.intoSInt**) of size **volumeWidth + 1** bits.
2. Perform signed multiplication between the 16-bit signed input sample and **volumeSigned** to produce a product of size **16 + volumeWidth + 1** bits.
3. Scale the product down by shifting it right by **volumeWidth** bits, and resize back to 16 bits: **scaledSample** = **(product >> volumeWidth).resize(16 bits)**
4. Register both the scaled payload and valid signal for a 1-cycle pipeline latency.

8. Decimator

IO Bundle

```
val io = new Bundle {
  val sampleTick = in Bool()
  val sampleIn   = slave(Flow(SInt(16 bits)))
  val sampleOut  = master(Flow(SInt(16 bits)))
}
```

Responsible for converting the 480 kHz oversampled waveform stream into 48 kHz audio samples. The decimator captures every 10th oscillator sample.

sampleOut is:

- registered
- stable
- updated only at 48 kHz

valid communicates that a new 48 kHz sample is available now.

9. I2STransmitter

IO Bundle

```
val io = new Bundle {  
    val sampleIn = slave(Flow(SInt(16 bits)))  
    val bclk      = out Bool()  
    val lrclk     = out Bool()  
    val sdata     = out Bool()  
}
```

Self-contained serializer and protocol engine.

Responsibilities:

- BCLK generation
- LRCLK generation
- shift register control
- stereo frame timing
- serial audio output

The module operates directly from the 24 MHz master clock.

The serializer uses the scheduled timing subpattern:

```
16, 16, 15, 16, 16, 15, 16, 15
```

to generate the required average I²S bit timing.

9.1 Gated Startup and Idle State

The transmitter employs a gated startup mechanism to ensure that the I2S clock and data lines only toggle when valid audio data is present.

Idle Behavior: The transmitter starts in an inactive state after reset:

- `bclk` and `sdata` are held **Low**.
- `lrclk` is held **High** (the standard idle state for I2S).

Activation: The state machine transitions to **active** upon the first assertion of `io.valid`. On this cycle:

- Internal counters (`bitCounter`, `cycleCounter`, `patternIndex`) are reset to 0.
- The input sample is latched into the `sampleBuffer`.
- The `shiftRegister` is loaded, and transmission of the Left channel begins immediately.

Once active, the transmitter remains in the active state to maintain a continuous bit clock, even if subsequent **valid** pulses are delayed, though it will re-synchronize its frame boundaries to the **valid** signal to prevent drift.